In [ ]:
```
'Welcome'
```

Out[ ]:
```
'Welcome'
```

In [1]:
```
a = 2
b= 4
c = a + b
print(c)
```

```
6
```

In [ ]:
```
test = 1
test
print(test)
```

In [1]:
```
nit = 1010
nit
```

Out[1]:
```
1010
```

In [2]:
```
nit = 1020
NIT
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 2
      1 nit = 1020
----> 2 NIT

NameError: name 'NIT' is not defined
```

In [3]:
```
nit1 = 1030
nit1
```

Out[3]:
```
1030
```

In [4]:
```
1nit = 2040
1nit
```

```
  Cell In[4], line 1
    1nit = 2040
    ^
SyntaxError: invalid decimal literal
```

In [5]:
```
nit@ = 100
nit@
```

```
  Cell In[5], line 1
    nit@ = 100
         ^
SyntaxError: invalid syntax
```

In [6]:
```
_nit = 4050
_nit
```

Out[6]: 4050

In [7]:
```
true = 4
true
```

Out[7]: 4

In [8]:
```
True = 5
True
```

```
  Cell In[8], line 1
    True = 5
    ^
SyntaxError: cannot assign to True
```

In [9]:
```
_
```

Out[9]: '{"dataframes": [], "user": "duggi"}'

In [10]:
```
_nit = ramesh
_nit
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 _nit = ramesh
      2 _nit

NameError: name 'ramesh' is not defined
```

In [11]:
```
_nit = "ramesh"
_nit
```

Out[11]: 'ramesh'

In [12]:
```
_
```

Out[12]: '{"dataframes": [], "user": "duggi"}'

In [18]:
```
_nit = 5
_nit
```

Out[18]: 5

In [16]:

Out[16]: '{"dataframes": [], "user": "duggi"}'

In [19]:
```
import keyword
keyword.kwlist
```

Out[19]:
```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

1020 = nit nit

In [24]:
```
i = 5
i
type(i)
id(i)
```

Out[24]:
```
140716634973224
```

f = 110.45 f

In [26]:
```
f = 290.05
f
type(f)
```

Out[26]:
```
float
```

In [28]:
```
gold_price = 14800.78
gold_price
type(gold_price)
```

Out[28]: float

In [ ]:

In [33]:
```
10 + 5
10 - 5
10 * 5
```

Out[33]: 50

In [34]:
```
print(10 + 5)
print(10 - 5)
print(10 * 5)
```

```
15
5
50
```

In [ ]:

In [ ]:

In [36]:
```
f1 = 2.4e1
f1
type(f1)
```

Out[36]: float

In [37]:
```
f2 = 3.5e3
f2
```

Out[37]: 3500.0

In [38]:
```
f3 = 3.2E3
f3
```

Out[38]: 3200.0

In [ ]:
```
# boolean value
```

In [39]:
```
True
```

Out[39]: True

In [40]:
```
False
```

Out[40]: False

In [41]:
```
True + True
```

Out[41]: 2

In [42]:
```
True + False - False
```

Out[42]: 1

In [43]:
```
True * True
```

Out[43]: 1

In [45]:
```
False / True #float division
```

Out[45]: 0.0

In [46]:
```
False // True
```

Out[46]: 0

In [47]:
```
True / False
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[47], line 1
----> 1 True / False

ZeroDivisionError: division by zero
```

In [48]:
```
int(True)
```

Out[48]: 1

In [49]:
```
int(False)
```

Out[49]: 0

# Complex ---> no other programming language has complex data type except python

# Complex --> real part & imaginary part

In [50]:
```
c = 10 + 20j # 10 - real part & 20 imagenary part where j = square root  of -1
c
```

Out[50]: (10+20j)

In [51]:
```
type(c)
```

Out[51]:
```
complex
```

In [52]:
```
c = 10 + 20j
c
```

Out[52]:
```
(10+20j)
```

In [53]:
```
c.real
```

Out[53]:
```
10.0
```

In [54]:
```
c.imag
```

Out[54]:
```
20.0
```

In [55]:
```
b = True
b1 = False
c = b + b1
c
```

Out[55]:
```
1
```

In [56]:
```
b = False
type(b)
```

Out[56]:
```
bool
```

In [57]:
```
c = 1 + 2j
c1 = 3 + 4j
print(c)
print(c1)
```

```
(1+2j)
(3+4j)
```

In [58]:
```
c + c1
```

Out[58]:
```
(4+6j)
```

In [ ]:
```
#string
```

In [59]:
```
'''welcome naresh it '''
```

Out[59]:
```
'welcome naresh it '
```

In [60]:
```
"welcome naresh it"
```

Out[60]:
```
'welcome naresh it'
```

In [61]:
```
s = 'python'
s
```

Out[61]: 'python'

In [62]:
```
s1 = 'welcome to genai agentic ai'
s1[3]
```

Out[62]: 'c'

In [63]:
```
s1[5]
```

Out[63]: 'm'

In [64]:
```
s1[11]
```

Out[64]: 'g'

In [65]:
```
s1[78]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[65], line 1
----> 1 s1[78]

IndexError: string index out of range
```

In [66]:
```
s[-3]
```

Out[66]: 'h'

In [67]:
```
s2 = 'nareshtechnology'
s2
```

Out[67]: 'nareshtechnology'

In [69]:
```
s2[:]
```

Out[69]: 'nareshtechnology'

In [70]:
```
len(s2)
```

Out[70]: 16

In [71]:
```
s2[3:]
```

Out[71]: 'eshtechnology'

In [73]:
```
s2[0:]
```

Out[73]: 'nareshtechnology'

In [74]:
```
s2[:6]
```
Out[74]: 'naresh'

In [75]:
```
s2[:100]
```
Out[75]: 'nareshtechnology'

In [76]:
```
s3 = 'welcometeam'
s3
```
Out[76]: 'welcometeam'

In [77]:
```
s3[1:5]
```
Out[77]: 'elco'

In [78]:
```
s3[1:8:3]
```
Out[78]: 'eot'

In [79]:
```
s3
```
Out[79]: 'welcometeam'

In [80]:
```
s3[::1]
```
Out[80]: 'welcometeam'

In [81]:
```
s3[::2]
```
Out[81]: 'wloeem'

In [82]:
```
s3[::-1]
```
Out[82]: 'maetemoclew'

In [83]:
```
s3[::-2]
```
Out[83]: 'meeolw'

In [84]:
```
s3[2:5]
```
Out[84]: 'lco'

# python typecasting

In [ ]:
```
- car()  -- car function without argument
- asian_cars('tata','toyota')  -- car function with 2 arguments | type of argument
- german_cars('audi','bmw','ferarai','vw') - germ car with 4 arguments
```

# we will type cast all other data type to integer

In [85]:
```
int(2.3)
```

Out[85]:
```
2
```

In [86]:
```
int(2.3, 4.5)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[86], line 1
----> 1 int(2.3, 4.5)

TypeError: 'float' object cannot be interpreted as an integer
```

In [87]:
```
int(True)
```

Out[87]:
```
1
```

In [88]:
```
int(1 + 2j)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[88], line 1
----> 1 int(1 + 2j)

TypeError: int() argument must be a string, a bytes-like object or a real number, no
t 'complex'
```

In [89]:
```
int('10')
```

Out[89]:
```
10
```

In [90]:
```
int('ten')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[90], line 1
----> 1 int('ten')

ValueError: invalid literal for int() with base 10: 'ten'
```

# other data types to float

In [91]:
```
float(20)
```

Out[91]:
```
20.0
```

In [92]:
```
float(True)
```

Out[92]:
```
1.0
```

In [93]:
```
float(False)
```

Out[93]:
```
0.0
```

In [94]:
```
float('20')
```

Out[94]:
```
20.0
```

In [95]:
```
float('ten')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[95], line 1
----> 1 float('ten')

ValueError: could not convert string to float: 'ten'
```

In [96]:
```
float(1+2j)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[96], line 1
----> 1 float(1+2j)

TypeError: float() argument must be a string or a real number, not 'complex'
```

# all other datatypes to bool

In [97]:
```
bool(2)
```

Out[97]:
```
True
```

In [98]:
```
bool(0)
```

Out[98]:
```
False
```

In [99]:
```
bool(4)
```

Out[99]:
```
True
```

In [100…
```
bool(1+2j)
```

Out[100…
```
True
```

In [101…
```
bool('10')
```
Out[101…
```
True
```

In [102…
```
bool('ten')
```
Out[102…
```
True
```

In [103…
```
bool( )
```
Out[103…
```
False
```

In [104…
```
bool(2.3)
```
Out[104…
```
True
```

In [105…
```
bool(0)
```
Out[105…
```
False
```

In [ ]:
```

```

In [106…
```
val1 = 20
val1
```
Out[106…
```
20
```

In [107…
```
import sys
sys.getsizeof(val1)
```
Out[107…
```
28
```

In [116…
```
val1 = [20 ,35 ,40, 60, 45,29,14]
val1
```
Out[116…
```
[20, 35, 40, 60, 45, 29, 14]
```

In [113…
```
sys.getsizeof(val1)
```
Out[113…
```
88
```

In [117…
```
sys.getsizeof(val1)
```
Out[117…
```
120
```

In [ ]:
```

```